Data Visualization Homework
Task 1 (Japan immigration) & Task 2 Part A (Whoop dataset analysis)

Mustafa Bozyel 32417

March 23, 2026

GitHub repository: https://github.com/mufasa-349/Data-Visualization-HW2.git

Abstract

This report has two parts. **Task 1** covers building an interactive Python dashboard for Japan inbound immigration statistics, a Gestalt/Bertin analysis of the interface, and screenshots before and after UI revision (with **Cursor** as the AI coding assistant). **Task 2 Part A** analyzes the `whoop_fitness_dataset_100k.csv` file: what the columns mean, and which attributes would be most useful in a mobile fitness app and why. **Task 2 Part B** presents three paper sketches plus a D3.js prototype (Figure 5), with Gestalt/Bertin-based design notes tied to the CSV attributes.

Part I
Task 1 — Japan immigration dashboard

1 What we did (overview)

1.1 Task A — Interactive dashboard

Original Cursor prompt (exact).

```
Cursor prompt:

A-) Download the Japan Inbound Immigration data from Kaggle. Create an interactive dashboard
    with your favorite AI.

This is my first task in the homework. @japan_immigration_statistics_inbound.csv I have
    already provided you with the data. I want you to create an interactive dashboard using
    python. nalayze and understand the data carefully. I have created a venv and i will
    install requirements there after you create the gui. Make it in english
```

- **What:** We visualized the dataset `japan_immigration_statistics_inbound.csv` (Japan inbound immigration, annual records from 1950–2005) using an interactive web dashboard.
- **How:** We used **Python** with **Streamlit** for layout, filters, and metrics, and **Plotly** for charts (time series, stacked areas by continent, country rankings, and a heatmap). Dependencies are listed in `requirements.txt` (`streamlit`, `pandas`, `plotly`). A virtual environment is used for installation.

- **AI-assisted coding (Cursor):** Implementation files—including `dashboard.py`, `requirements.txt`, and `.streamlit/config.toml`—were written and revised with **Cursor** acting as the **AI agent** for code generation: the assistant produced initial layouts, Plotly figures, theme helpers, bug fixes (e.g., compatibility with Plotly's API), and Streamlit API updates. The author reviewed, ran, and validated all code locally.
- **Deliverables:** Runnable application (`dashboard.py`), configuration (`.streamlit/config.toml` for theme), data file, and screenshots (`task1-a.png`, `task1-b.png`).

1.2 Task B — Gestalt & Bertin analysis

- **What:** We analyzed the dashboard using **Gestalt** grouping principles (proximity, similarity, continuity, figure–ground, common region) and **Bertin's** visual variables (especially position, size, color/value, and redundant encoding).
- **How:** Findings were written in `task1-b-gestalt-bertin-analysis.txt`. The same criteria guided UI revisions in `dashboard.py`.
- **Outcome:** The report below merges that analysis with a **before/after** narrative: Figure 1 reflects the pre-revision UI; Section 4 summarizes changes aligned with the checklist; Figure 2 shows the revised dashboard; Section 6 states explicitly *what is better* after revision and why.

1.3 Task C — LaTeX homework report (prompt)

Original Cursor prompt (exact).

```
I want you to crete a .tex file for this homework. Include what how and whtt we have done
    stuff. Also include @task1-a.png this image before principles and include revised verion
    @task1-b.png . Add following infos to latex file @task1-b-gestalt-bertin-analysis.txt that
    includes analysis results for before and after gestlat and bertin principles revision.
    Make report english as well
```

2 Dashboard screenshot (before principles-based revision)

Figure 1 shows the initial dashboard (Time series tab), which was used as the basis for the Gestalt/Bertin critique. At this stage, the layout already used a dark theme, sidebar filters, KPI row, and two charts (annual total line chart and year-over-year percentage bars).

3 Gestalt and Bertin analysis (baseline — before revision)

This section summarizes the written analysis in `task1-b-gestalt-bertin-analysis.txt`, describing the **pre-revision** interface.

3.1 Gestalt — strengths

- **Proximity:** Filters are grouped in the sidebar; KPI metrics sit in one row, so controls and summary numbers form coherent units.
- **Similarity:** Both charts use the same hue family (blue) for data, signaling one phenomenon (immigration counts and their change).
- **Continuity:** The line chart supports a clear temporal reading from 1950 to 2005.
- **Figure–ground:** On a dark background, light chart elements and text separate well from the panel.

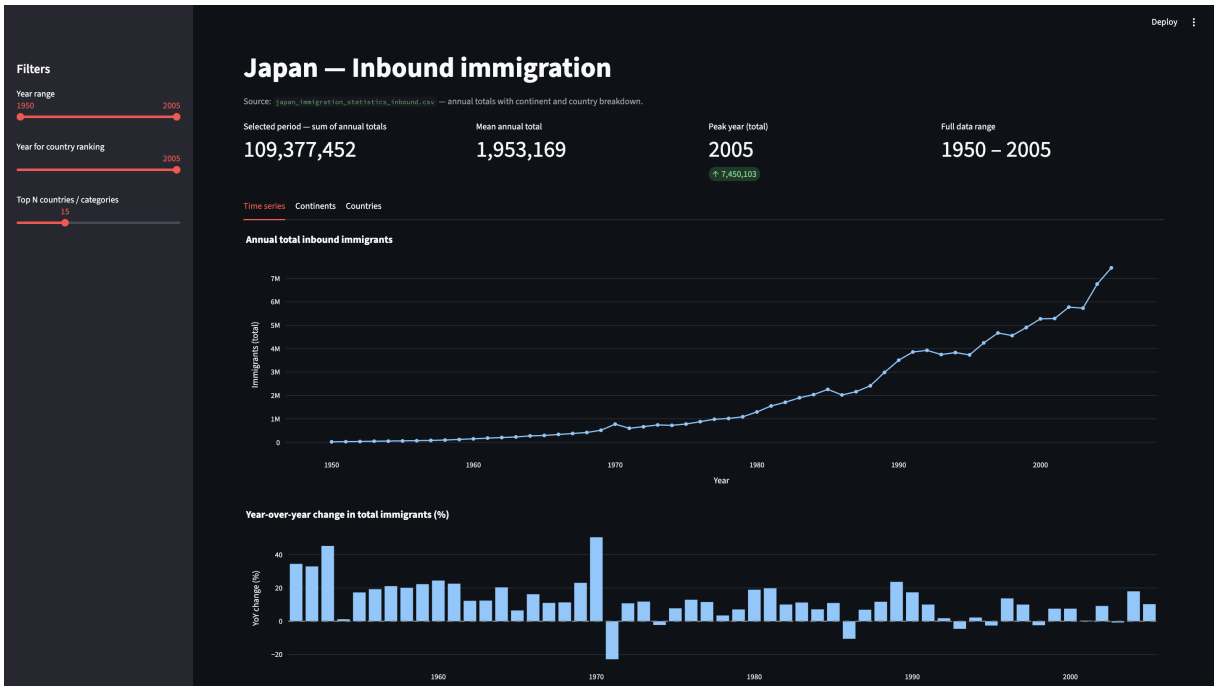


Figure 1: Initial interactive dashboard (before revision): Japan inbound immigration — Time series view with KPIs, annual total line chart, and YoY change bar chart.

- **Common region:** The main content is visually separated from the sidebar (controls vs. readout).

3.2 Gestalt — weaknesses

- **Similarity / semantics:** Red accents (sliders, active tab) vs. green KPI "delta" mix navigation and magnitude cues; users may briefly confuse "red" with negative data.
- **Continuity / YoY:** The YoY chart omits the first year in the filtered range (no prior row); a short caption was missing.
- **Proximity:** Four KPIs in one row can feel dense on narrow layouts; definitions were not always explicit.
- **Closure:** The total series could use a subtle area under the line to reinforce cumulative "volume" over time.

3.3 Bertin — strengths

- **Position:** Time on the horizontal axis and magnitude on the vertical axis follows effective encoding priority.
- **Size (typography):** Title, KPI numerals, and axis ticks establish a reasonable hierarchy.
- **Color value:** Light blue on dark background works for the main series and bars.

3.4 Bertin — weaknesses

- **Redundant encoding:** YoY sign was carried mainly by position above/below zero; hue could reinforce positive vs. negative years.
- **Mark size / noise:** Markers at every year on the line chart add visual noise on long series.

- **Units and definitions:** Axis titles and subtitles should state units and the meaning of `total` explicitly.
- **Contrast:** Axis tick labels on dark theme should meet comfortable contrast (WCAG-oriented).

3.5 Summary (pre-revision)

Strengths included clear temporal layout (Bertin: position), coherent grouping (Gestalt: proximity), and strong figure–ground separation. Gaps included redundant encoding for signed YoY change, clearer semantics for accent colors, and richer annotations (units, first-year YoY behavior).

4 Principles-based revisions (what we implemented)

Table 1 maps the actionable checklist from the analysis file to concrete changes in code and configuration.

Principle / goal	**Implementation**
Chart subtitles & definition of `total`	Subtitles under chart titles (Plotly `title` HTML); axis labels such as "Immigrants (count per year)".
YoY redundant encoding (hue + position)	Bar color by direction (increase / decrease / unchanged) with a distinct palette; clearer zero line.
WCAG-friendly contrast on dark theme	Central `apply_dashboard_theme()`: tick color `#B0B3B8`, titles `#E8EAED`, subtle grids.
Consistent styling across figures	Shared theme function for line, bar, area, and heatmap; `plotly_dark` template and transparent plot backgrounds.
Document units everywhere	Metric `help=` tooltips; captions explaining YoY on filtered rows; heatmap colorbar title.
Closure / volume for the total series	Filled area under the line via `go.Scatter` with `fill=tozeroy` (avoids `px.line` lacking `fill` in some Plotly versions).
Reduce line-chart noise	Line-only trace (no per-year markers) for the annual total series.
Streamlit chrome vs. data colors	`.streamlit/config.toml`: dark base theme and primary accent aligned with chart cyan (`#38BDF8`) to reduce clash with KPI delta green.
Deprecation fix	`st.plotly_chart(..., width="stretch")` instead of `use_container_width`.

Table 1: Mapping from analysis checklist to implemented revisions.

5 Dashboard screenshot (after revision)

Figure 2 shows the revised dashboard. Visually, the total series now includes a filled area under the line; YoY bars use color to encode sign; typography and grids are harmonized; and textual annotations (subtitles, captions, metric help) clarify definitions and the first-year YoY behavior.

6 What is better after principles integration and revision?

The revisions were not only cosmetic: they address the gaps identified in Section 3 and make the dashboard **easier to read**, **less ambiguous**, and **more accessible** on a dark theme. Table 1

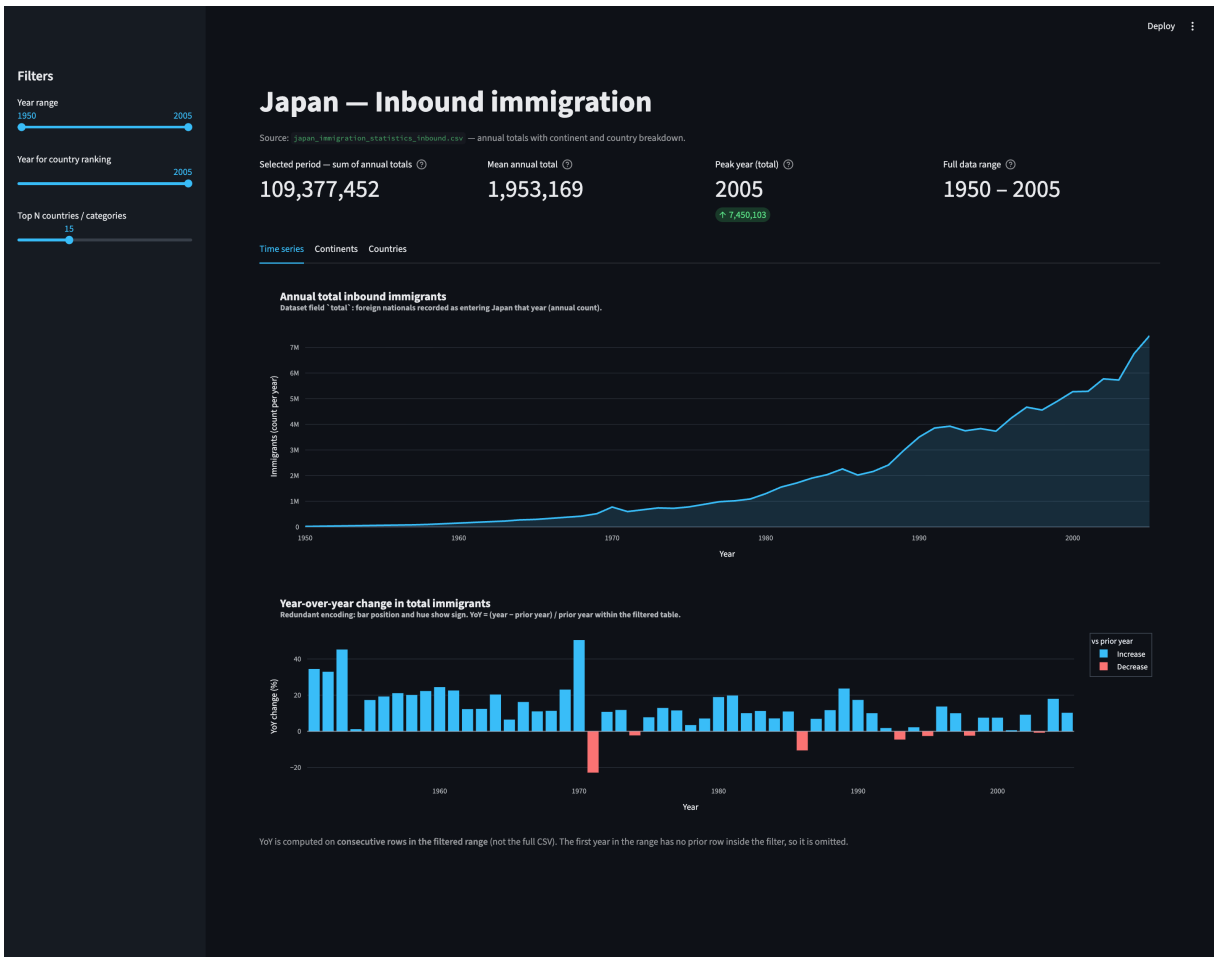


Figure 2: Revised interactive dashboard (after Gestalt/Bertin–aligned changes): unified theme, filled area under the annual total, redundant hue encoding for YoY bars, improved axis/tick contrast, and explanatory subtitles/captions.

lists *what* changed; here we state *why* the post-revision interface is stronger.

6.1 User-visible improvements (benefits)

- **Faster reading of the main trend (Gestalt closure):** The filled area under the annual total reinforces "volume over time" at a glance; removing dense per-year point markers reduces clutter so the long-run pattern stands out.
- **Clearer year-over-year story (Bertin redundant encoding):** Positive and negative changes are distinguishable both by bar direction *and* by color, so users do not rely on a single channel; the zero baseline is visually sharper.
- **Less confusion between UI chrome and data (Gestalt similarity):** Aligning Streamlit's primary accent with the chart palette (`config.toml`) reduces the chance that control colors are mistaken for "bad" or "negative" data, compared with a stronger mismatch between red UI accents and green KPI deltas.
- **Explicit meaning of numbers (annotation):** Chart subtitles, axis titles with units (*count per year*), KPI tooltips, and the YoY caption explain what `total` means and why the first filtered year has no YoY bar—reducing misinterpretation.
- **More comfortable reading on dark backgrounds (contrast):** Systematic tick and title colors in `apply_dashboard_theme()` improve legibility of axes and legends relative to an ad hoc default styling.
- **Coherent visual language across tabs:** One theme function and shared palette logic (continents, heatmap colorbar) make the dashboard feel like one instrument rather than unrelated charts.

6.2 Mapping principles to outcomes

- **Gestalt:** Revisions strengthened **similarity** (UI vs. data color harmony), **closure** (area fill for totals), and **proximity of meaning** (subtitles and help near the metrics they explain). Layout grouping was already sound and was preserved.
- **Bertin: Position** for time and magnitude was kept; **redundant encoding** was added for YoY sign; **color value** and tick contrast were standardized; unnecessary **mark** noise on the line chart was removed. Definitions and units are now tied explicitly to encodings (labels and tooltips).

7 Conclusion (Task 1)

We built an interactive Japan inbound immigration dashboard in Python (Streamlit + Plotly), analyzed it with Gestalt and Bertin lenses, and applied a focused set of revisions. After integration of these principles, the interface improves in **interpretive clarity** (what each number means), **encoding redundancy** for signed change (YoY), **visual continuity** for the level series (area fill), **legibility** (contrast and typography), and **consistency** across charts and the Streamlit shell. The screenshots `task1-a.png` and `task1-b.png` document the interface before and after these changes; Section 6 summarizes concrete benefits of the revised version.

Part II
Task 2 — Whoop-style fitness dataset

8 Task 2 Part A: Analysis of the Whoop health dataset (for a mobile app)

This section looks at `whoop_fitness_dataset_100k.csv`. The file has one row per user per day (about 100k rows). It is built like a **wearable / Whoop-style** daily log: sleep, recovery, strain, workouts, and heart metrics. Below I explain what is in the data and what I would actually show in a **phone app** and why.

8.1 What the dataset contains (big picture)

Each row is a **single day** for one `user_id`. You get the date, day of week, some **profile** fields (age, gender, height, weight), and then numbers that describe **readiness**, **load**, **sleep**, and **activity**. Workout-related columns are filled when the person trained; on rest days the activity type is often "Rest Day" and duration/strain can be zero.

8.2 Column groups (simple overview)

- **Time and identity:** `user_id`, `date`, `day_of_week` — needed to sort days and build a calendar or timeline in the app.
- **Profile:** `age`, `gender`, `weight_kg`, `height_cm`, `fitness_level`, `primary_sport` — good for onboarding screens, goals, and simple personalization (not something that changes every hour).
- **Daily readiness / load ("Whoop-like" scores):** `recovery_score`, `day_strain` — these are the numbers users often want **on the home screen**: "how ready am I?" vs. "how hard was today?".
- **Sleep:** `sleep_hours`, `sleep_efficiency`, `sleep_performance`, `light_sleep_hours`, `rem_sleep_hours`, `deep_sleep_hours`, `wake_ups`, `time_to_fall_asleep_min` — perfect for a **sleep tab**: total time, quality, and stage breakdown.
- **Heart and stress signals:** `hrv`, `resting_heart_rate`, `hrv_baseline`, `rhr_baseline`, `respiratory_rate`, `skin_temp_deviation` — useful for "recovery" and health trends; comparing today to **baseline** helps users see if they are off their normal (tired, sick, etc.).
- **Energy:** `calories_burned` — easy to show as a daily ring or total, similar to other fitness apps.
- **Workout block:** `workout_completed`, `activity_type`, `activity_duration_min`, `activity_strain`, `avg_heart_rate`, `max_heart_rate`, `activity_calories`, `hr_zone_1_min`–`hr_zone_5_min`, `workout_time_of_day` — detail screen after a session: what sport, how long, how hard, and time in heart-rate zones.

8.3 Which attributes are most useful in a mobile app (and why)

- **`recovery_score` + `day_strain`:** Very useful. On a small screen you want **two clear numbers** that answer: "Can I train hard today?" and "Did I already load my body?" They match how real wearables market their product, so users understand them quickly.

- **Sleep totals and stages (`sleep_hours`, efficiency, light/REM/deep):** Very useful. People open health apps in the morning to see **how long** they slept and **sleep quality**. Stages help users see if deep/REM was short even when total hours look okay.
- **`hrv` and `resting_heart_rate` with baselines:** Useful for a "recovery" or "body status" card. If today's HRV is far from `hrv_baseline` or RHR is off `rhr_baseline`, the app can show a gentle warning (rest day, not a medical diagnosis—just trend).
- **`calories_burned`:** Useful for users who care about energy balance; easy to show as one number or a progress ring.
- **Workout summary (`activity_type`, `activity_duration_min`, `activity_strain`, HR avg/max, `activity_calories`):** Essential on the **workout detail** page. The user wants to see what they did and how hard it was without scrolling forever.
- **Heart-rate zones (`hr_zone_1_min`–`hr_zone_5_min`):** Useful for people who train with intensity, but on mobile it is better to show a **small bar chart or one line of text** (e.g. "12 min hard") instead of five raw numbers, so the screen does not feel crowded.
- **`workout_time_of_day`:** Nice for habits—morning vs. evening training—and for simple insights ("you usually work out in the evening").
- **Profile fields (`fitness_level`, `primary_sport`):** Good for recommendations and filtering (beginner vs. advanced), not for daily graphs.

8.4 Attributes that are less critical on the phone (or need simplification)

- **Five separate HR zone columns:** Important for training nerds, but the app should **aggregate** them for the main view (e.g. total moderate + hard minutes).
- **`skin_temp_deviation` and `respiratory_rate`:** Still valuable for wellness, but many users will not look at them every day—better in a "health details" or weekly summary.
- **`wake_ups` and `time_to_fall_asleep_min`:** Good for sleep geeks; keep them on the sleep detail page, not on the home screen.

8.5 Things to be careful about (student-level notes)

- The data is **not real medical data** from our class project point of view—treat it as a **sample** for building UI and charts.
- On **rest days**, workout fields are often zero or "N/A"; the app must handle that so it does not look like a bug.
- **Privacy:** a real app would need strong rules for `user_id` and health fields; we only talk about **which columns are useful for design**, not about storing real patient data.

8.6 Short summary

For a mobile app, I would prioritize **recovery**, **strain**, **sleep summary**, **HRV/RHR vs. baseline**, and **workout type + duration + strain + calories**. Heart-rate zones and extra sleep metrics are great on a second screen. Profile fields support onboarding and personalization. This matches what users expect from a Whoop-style experience: **simple daily scores first**, details when they tap.

9 Task 2 Part B: Mobile dashboard sketches (Whoop-style CSV)

This section shows **three** paper sketches and the **D3.js** implementation (`d3js.html`) captured in `d3js-design.png`. All follow the Task 2 Part A column choices and use Gestalt/Bertin ideas for small screens.

9.1 Sketch 1: `sketch1.jpeg`

9.2 Mobile dashboard layout (four cards)

The dashboard is split into four main **cards** the user would see when opening the app.

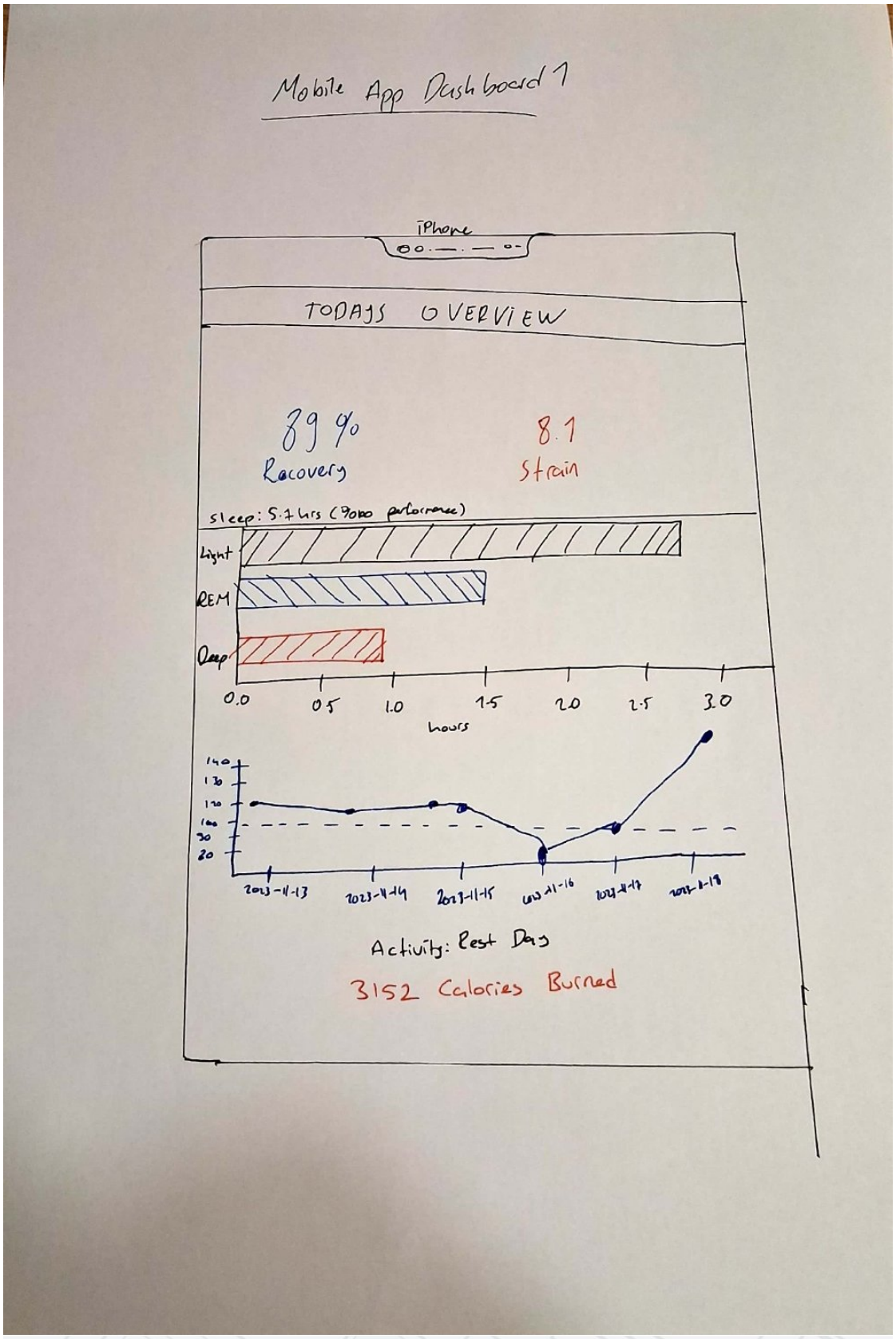
- **Daily overview ("Ready to train?")**: The most important metrics—**Recovery Score** and **Day Strain**—sit at the top. They map to `recovery_score` and `day_strain` in the CSV.
- **Sleep analysis**: Shows sleep duration and quality with a **stacked** view of stages (light, REM, deep). This uses `sleep_hours`, `sleep_efficiency`, and stage columns such as `light_sleep_hours`, `rem_sleep_hours`, `deep_sleep_hours`.
- **Recovery trends**: A **line chart** of HRV over several days compared to the user's **baseline**. This ties to `hrv`, `hrv_baseline`, and related day-by-day rows.
- **Activity summary**: Daily calories and **heart-rate zones** (time in each zone). This uses `calories_burned` and `hr_zone_1_min`–`hr_zone_5_min`.

9.3 Gestalt principles (how the sketch uses them)

- **Proximity:** Numbers are grouped into clear cards. Sleep hours, efficiency, and stages stay in one block so the user reads them as one "sleep" unit.
- **Similarity:** One color family (blue shades) for sleep stages (Deep, REM, Light) so they are seen as parts of the same sleep story.
- **Continuity:** The HRV chart is a connected line so the eye follows the trend over time (up or down vs. scattered dots).
- **Figure–ground:** Large, high-contrast numbers for Recovery and Strain (e.g. "89%") stand out as the main "figure" on a calm background—quick read in under a second.

9.4 Bertin's visual variables (Semiology of Graphics)

- **Position:** Vertical **order** = importance (best scores at the top). Horizontal **time** on the HRV strip = last 7 days.
- **Size:** Recovery and Strain digits are larger than other text, so the user knows what matters most for daily decisions.
- **Value / color:**
  - **Qualitative (status):** Green for high recovery, orange/red for low—fast status read.
  - **Sequential (intensity):** HR zone chart uses a yellow–to–red gradient: darker/warmer = harder effort.
- **Shape / texture:** Solid line = current HRV; dashed line = baseline. Different texture so "today" vs. "normal" is clear without a heavy legend.



Scanned with CamScanner

Figure 3: Mobile dashboard sketch 1: Whoop-style health overview using CSV concepts (recovery, strain, sleep, HRV trend, HR zones).

9.5 Data mapping summary (what encodes what)

- **Recovery & Strain:** size and color (color = quick status).
- **Sleep stages:** bar length + blue hue steps (composition of the night).
- **HRV & RHR: position** on a time axis (trend vs. baseline).
- **HR zones:** bar length + saturation (redder = harder zone).

9.6 Sketch 2: Dark theme & advanced metrics (`sketch2.jpeg`)

Sketch 2 is a **dark-mode** layout aimed at high contrast in low light. It pulls in **more columns** from the CSV than sketch 1: breathing and skin temperature sit next to heart metrics, and baselines appear next to live values.

New features (data mapping).

- **Health monitor card:** Combines `respiratory_rate` and `skin_temp_deviation` with HR-style signals so the user sees a wider "internal state" in one place.
- **Baseline comparisons:** Today's HRV/RHR are shown with `hrv_baseline` and `rhr_baseline` as reference (small text or subscript). Bertin: **position** and alignment make "now vs. normal" easy to compare without a second chart.
- **Intensity (HR zones):** Uses `hr_zone_1_min` through `hr_zone_5_min` in a horizontal bar—left-to-right matches "easy → hard" reading.
- **Activity block:** `activity_type`, duration, avg/max HR, and `activity_calories` with a bar so calories read as **length** (Bertin: size for quantity).

Gestalt (short). **Closure:** Each topic lives in its own card (health vs. activity) so groups are clear without thick lines. **Figure–ground:** Neon-style greens/blues/reds on near-black make numbers jump forward. **Similarity:** Repeated pattern Value → Label → Baseline in the health strip teaches one rhythm. **Proximity:** Duration, avg HR, and max HR stay together as one workout event.

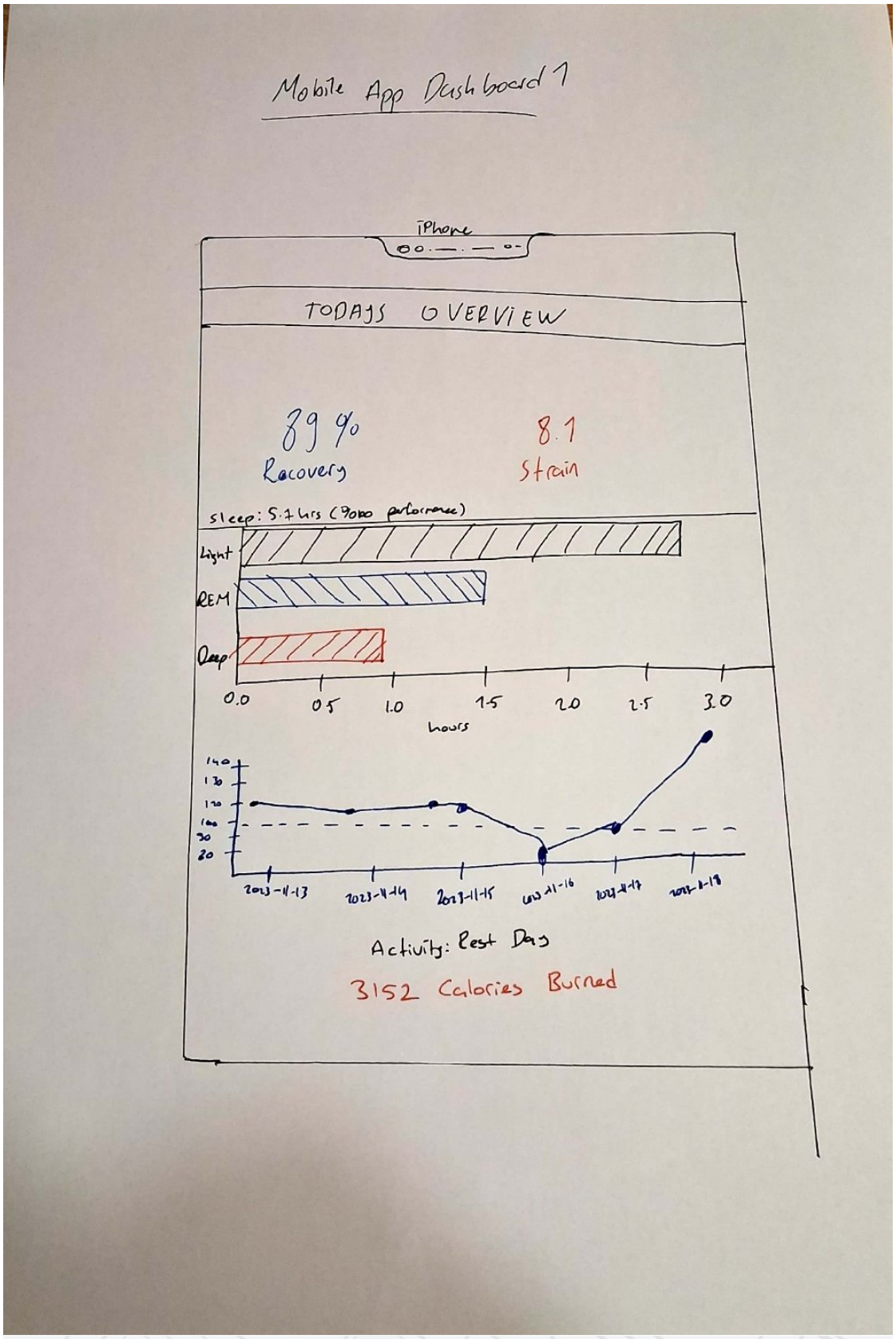
Bertin (short). Recovery/Strain stay **larger** than the monitor tiles (global vs. supportive metrics). **Hue** on zones runs green→yellow→orange→red for aerobic→anaerobic. **Orientation:** Horizontal zones save vertical space for labels. **Position:** Readiness block stays at the **top** (highest priority).

9.7 Sketch 3 and D3.js prototype (`sketch3.jpeg`, `d3js-design.png`)

Sketch 3 closes the set of three concepts. The screenshot `d3js-design.png` shows how the same idea is implemented with **D3.js** (concentric rings, zone bar, metric cards). Below is a short list of what we improved on purpose.

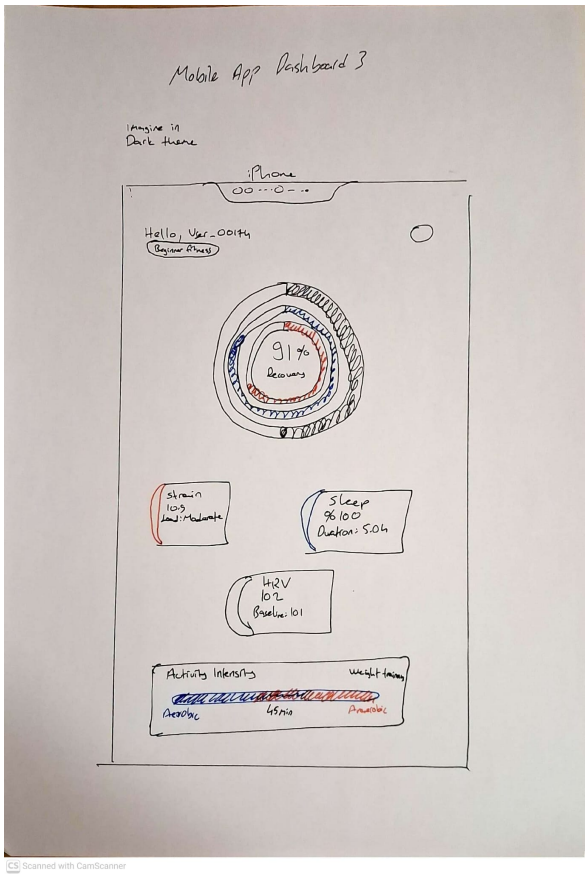
Key design enhancements

- **Bertin — size and orientation:** The **orbit** uses concentric circles: different **radius** = different metrics (Strain, Sleep, HRV, RHR), so the user can read "one system" at a glance. The **zone bar** uses a **linear** (horizontal) layout: left-to-right shows how much time fell in aerobic vs. anaerobic HR zones, so **proportion** of the workout is easy to see.



Scanned with CamScanner

Figure 4: Mobile dashboard sketch 2: dark theme (#0F0F0F), extra vitals (`respiratory_rate`, `skin_temp_deviation`), baselines as context, HR zones, and activity detail.



(a) Paper sketch 3 (final layout concept).

(b) D3.js dashboard built in `d3js.html` (browser screenshot).

Figure 5: Third dashboard design and the matching D3.js prototype side by side.

- **Gestalt — proximity:** Health numbers sit in a **2×2 grid**. HRV and its baseline are **close together** on the same card, so the brain treats them as one idea: "current readiness vs. normal."
- **Color context (similarity):** Card **left-border** colors are not random: they **match** the color of the matching ring in the orbit. Same color for the same metric = same mental group (Gestalt **similarity**).
- **Health trends (common region):** On the HRV card, a small green "+1" (or similar delta) sits **next to** the main number. That groups the **change** with the **value** in one visual region, so the user reads "status + update" as one chunk (Gestalt **common region**).

AI tooling disclosure

Cursor (https://cursor.com) was used as the **AI coding agent** integrated into the editor for this assignment: it assisted with generating, editing, and debugging Python/Streamlit/Plotly code for the dashboard and project files. The LaTeX source for this report was also produced with AI assistance. The author reviewed all outputs, ran the application locally, and is responsible for the final text, figures, and interpretation.

Data and code

Task 1 data: `japan_immigration_statistics_inbound.csv`.
Task 1 main script: `dashboard.py`.
Task 1 analysis notes: `task1-b-gestalt-bertin-analysis.txt`.
Task 2 Part A dataset: `whoop_fitness_dataset_100k.csv`.
Task 2 Part B sketches: `sketch1.jpeg`, `sketch2.jpeg`, `sketch3.jpeg`.
D3.js prototype: `d3js.html`, screenshot `d3js-design.png`.